AutoBind

Export Flow

Click Export in the Prefab Auto Bind window to generate C# scripts for each export unit, including the root class and nested view classes.

- Logic output: `{LogicDir}/{ClassName}.cs`
  - Created only when the file does not exist.
  - Contains an empty `partial class` for handwritten logic.
- Bind output: `{PartialDir}/{ClassName}.Bind.cs`
  - Always overwritten.
  - Contains generated fields, lookup statements, event statements, and handler declarations.

DirectReference mode mounts the generated logic component on the prefab and fills serialized references. BindByFind units that inherit `MonoBehaviour` are also mounted after compilation, but their fields remain runtime-bound by templates.

Editor Settings

Field	Purpose
Namespace	Optional namespace for generated files.
Parent Class	Optional base class. DirectReference uses `MonoBehaviour` when this is empty.
Export Mode	Selects DirectReference or BindByFind generation.
Logic Dir	Output directory for handwritten logic files.
Partial Dir	Output directory for generated bind files.
Bind Template	Main code template that composes generated statement blocks.
Find Statement Template	Generates lookup statements for ordinary bound components and inserts them at `${find_statements}`.
Add Event Statement Template	Generates subscription statements and inserts them at `${add_event_statements}`.
Remove Event Statement Template	Generates unsubscription statements and inserts them at `${remove_event_statements}`.
Event Handler Name Template	Generates the handler method name exposed as `${handler_name}` in event statement templates.
Root Object Name	Valid C# parameter name inserted at `${root_object}` and used as the initial `${parent_obj}` in BindByFind mode.
Plain Child View Bind Template	Generates acquisition and initialization statements for every BindByFind child view, including `MonoBehaviour` views, and inserts them at `${find_statements}`.

Bind Template Placeholders

Multiline placeholders must appear on a line by themselves.

Placeholder	Generated content
`${usings}`	Required `using` directives.
`${fields}`	Generated field declarations.
`${find_statements}`	Combined output of Find Statement Template and Plain Child View Bind Template. Required in BindByFind mode.
`${add_event_statements}`	Output of Add Event Statement Template.
`${remove_event_statements}`	Output of Remove Event Statement Template.
`${event_handlers}`	Generated partial handler declarations.
`${class_name}`	Generated class name.
`${namespace}`	Configured namespace.
`${parent_class}`	Configured parent class.
`${root_object}`	BindByFind `GameObject` parameter name from Root Object Name.

The default BindByFind signature is `protected override void BindUI(GameObject ${root_object})`. With the default Root Object Name `root`, it generates `protected override void BindUI(GameObject root)`.

Event Statement Templates

Both Add Event Statement Template and Remove Event Statement Template support:

Placeholder	Meaning
`${event_obj}`	Bound field access expression.
`${event_name}`	Unity event property name, such as `onClick`.
`${handler_name}`	Handler method name rendered from Event Handler Name Template.

Default Add Event Statement Template:

```
AddEvent(${event_obj}, "${event_name}", ${handler_name});
```

Default Remove Event Statement Template:

```
${event_obj}.${event_name}.RemoveAllListeners();
```

Default Event Handler Name Template:

```
On${event_obj}${event_name}
```

Plain Child View Bind Template

Every BindByFind child view uses the Plain Child View Bind Template from its own Settings asset. This applies to both plain C# views and `MonoBehaviour` views.

Placeholder	Meaning
`${obj_name}`	Assignment target; includes `var` for list temporary variables.
`${obj_ref}`	Target reference without a declaration prefix.
`${class_name}`	Generated child view type.
`${obj_path}`	Located child `Transform` expression.

The default template creates a plain C# view and binds its located `GameObject`:

```
${obj_name} = new ${class_name}();
${obj_ref}.Bind(${obj_path}.gameObject);
```

For a `MonoBehaviour` child view, its generated component is attached to the prefab after compilation. The template still controls how the parent acquires and initializes it, for example:

```
${obj_name} = ${obj_path}.GetComponent<${class_name}>();
${obj_ref}.Initialize();
```

No child `Bind` or initialization method is called implicitly; the configured template is the complete behavior.

Find Statement Template

BindByFind component lookup statements support:

Placeholder	Meaning
`${obj_name}`	Target field name.
`${parent_obj}`	Nearest already-bound ancestor field or Root Object Name.
`${path}`	Path relative to `${parent_obj}`.
`${component}`	Component type name.

Default Find Statement Template:

```
${obj_name} = FindChild(${parent_obj}.transform, "${path}", typeof(${component}));
```

Notes

- Root class names default to the prefab asset name.
- DirectReference requires generated types to compile before references can be mounted. On first generation, allow Unity to compile and run Export again if immediate mounting is needed.